Assignment 3

Generative Modeling

Submission Deadline: March 20, 2025 | Total: 5 marks

1. Overview

This assignment gives you hands-on experience training generative models. You will choose one of two tracks based on your interests. Both tracks are worth the same marks and are equally challenging.

Total marks: 5 | One submission per student
Deadline: March 20, 2025
You must choose exactly one track and complete it fully
Do not submit zip file. Only submit code file and the report. All the output plot and screenshot need to show in report.

2. Choose Your Track

Read both descriptions below, then pick one. You do not need to justify your choice.

Track 1 — Image Generation

Train a VAE and a GAN on a small image dataset and compare their outputs.

Track 2 — Fine-Tuning a Language Model

Pre-train or fine-tune an encoder or encoder-decoder LLM using self-supervised learning on a text task.

3. Track 1 — Image Generation (VAE + GAN)

3.1 Background

In this track you will implement two generative model families (Variational Autoencoders (VAE) and Generative Adversarial Networks (GAN)) on a small image dataset. You will compare their training behavior and the visual quality of their generated images.

You may start from the demo code provided in class, your own implementation, or any standard library. If you use external code, cite it clearly in your report.

3.2 Dataset

Pick one of the following, or another open image dataset of similar scale (check with your instructor if unsure):

- CIFAR-10 — 60,000 32×32 color images across 10 classes
- Fashion-MNIST — 70,000 28×28 greyscale images of clothing items
- Flowers102 — 8,000+ images of 102 flower categories
- Caltech-101 — diverse object images, moderate size

Keep images at or below 64×64 pixels to keep training time reasonable on a laptop or free-tier GPU. Report your exact dataset, resolution, and train/val split.

3.3 Part A — Variational Autoencoder (VAE)

Implement an encoder-decoder with a Gaussian latent prior. Train using the ELBO loss.

Required results to include in your report:

- A grid of 16 reconstructions showing input images next to their reconstructed versions.
- A grid of 16 images sampled from the prior (decode random $z \sim N(0, I)$).
- A plot of training loss (total ELBO, KL term, and reconstruction term) over epochs.

3.4 Part B — Generative Adversarial Network (GAN)

Train a DCGAN (or a simpler variant) on the same dataset, same resolution, and same train/val split as Part A. Track both generator and discriminator losses.

Required results to include in your report:

- A grid of at least 9 images generated by the trained generator.
- A plot of generator loss and discriminator loss over training iterations.
- Fréchet Inception Distance (FID) computed against the validation set. Inception Score (IS) is optional but encouraged.
- A brief note on whether you observed mode collapse or training instability, and what you tried.

Tip: torchmetrics provides a ready-to-use FID implementation (torchmetrics.image.FrechetInceptionDistance). You do not need to implement it from scratch.

3.5 Comparative Analysis

Run both models on the same dataset, the same image resolution, and the same train/val split. Report these settings explicitly at the top of this section.

Fill in a comparison table similar to below in your report (mark N/A where a metric does not apply):

Model	FID	IS	PSNR / SSIM	Wall-clock	Params	Steps
VAE	[fill]	N/A	[fill]	[fill]	[fill]	[fill]
GAN (DCGAN)	[fill]	[fill]	N/A	[fill]	[fill]	[fill]

Follow the table with a short discussion covering:

- Training stability — which model was easier to train? Did either diverge or oscillate?
- Data efficiency — how many epochs before samples looked coherent?
- Compute cost — relate training time and parameter count to output quality.
- Characteristic artifacts — e.g., blurry VAE samples from the Gaussian decoder, GAN texture repetition or checkerboard patterns.

3.6 Grading Rubric — Track 1

Component	What we look for	Marks	Your score
VAE implementation	ELBO loss implemented correctly; training converges without NaN	1	
GAN implementation	Generator/discriminator train together; samples improve over time	1	
Figures & visualisations	Both required image grids present and readable; loss curves included	1	
Comparative discussion	Identifies at least two differences between VAE and GAN outputs; uses evidence from results	1	
Code quality & report	Code runs end-to-end; report is clearly written and references cited	1	

Partial credit is given at every level. You will not lose marks for imperfect image quality as long as your implementation is correct and your discussion is honest about what went wrong.

4. Track 2 — Encoder-Decoder Language Model

4.1 Background

In class demo showed encoder-only (BERT) and decoder-only (GPT) models. This track completes the picture with an encoder-decoder model, which combines both halves: the encoder reads the source sequence and the decoder generates the target sequence by attending to the encoder output. You will follow a three-stage pipeline: load a pretrained checkpoint, continue pre-training it with self-supervised learning on your chosen dataset, then fine-tune it on a labelled task.

4.2 Stage 1 — Dataset and Pretrained Checkpoint

Dataset

You can use any summarisation dataset. For example, the XSum dataset. It is a news summarization dataset where each example is a BBC article paired with a one-sentence summary.

Dataset: XSum | load with: from datasets import load_dataset; ds = load_dataset('xsum')
HuggingFace link: hf.co/datasets/EdinburghNLP/xsum
Use a small subset: 5,000–10,000 examples is enough for free Colab.
Input: full BBC article text | Target: one-sentence summary

Load a pretrained checkpoint

Load one of the following small pretrained encoder-decoder models from Hugging Face:

- google-t5/t5-small — general purpose seq2seq, easy to fine-tune for summarization
- facebook/bart-base — designed for denoising tasks, a natural fit for Stage 2

In your report, state which model you chose and print its total parameter count.

4.3 Stage 2 — Self-Supervised Pre-Training on Article Text

Before fine-tuning on labelled pairs, continue training the checkpoint on the raw article texts — without using any summaries. This adapts the model to BBC news-style language before it ever sees a label.

The idea: masked denoising

You already saw this technique in demo with BERT. The only difference here is that instead of predicting the masked tokens from a single encoder, you use the full seq2seq model:

- Input to the encoder: an article sentence with 15% of tokens randomly masked (exactly as in the BERT demo)
- Target for the decoder: the original clean sentence

```
Encoder input:  "The [MASK] announced new measures to [MASK] inflation."
    Decoder target: "The government announced new measures to combat
                                    inflation."
```

The model learns to reconstruct corrupted news text, which forces it to understand news vocabulary and sentence structure. This is essentially how BART was originally pre-trained.

Implementation — reuse what you already know

Step 1: Extract only the article texts from XSum (ignore the summaries entirely for this stage).
Step 2: Use DataCollatorForLanguageModeling(tokenizer, mlm=True, mlm_probability=0.15) — same collator as the BERT demo.
Step 3: Pass the masked article as the encoder input and the original article as the decoder labels.
Step 4: Train for 500–1000 steps and save this checkpoint — it becomes Model A in Stage 3.

Required result: a plot of the SSL training loss over steps.

4.4 Stage 3 — Fine-Tuning and Comparison

Fine-tune two separate models on the labelled XSum pairs (article → one-sentence summary) and compare them:

	Starting point	Description
Model A	Stage 2 checkpoint	Fine-tune the SSL pre-trained (denoised) model from Stage 2
Model B	Stage 1 checkpoint	Fine-tune the original pretrained checkpoint directly (no SSL step)

Both models must use the same labelled examples, the same hyperparameters, and the same number of fine-tuning epochs. The only difference is the starting checkpoint.

Required results

- Fine-tuning loss curves for Model A and Model B on the same plot (or side by side).

- ROUGE-1 and ROUGE-L scores on the validation set for both models.
- At least three example outputs: for each example show the article snippet, the reference summary, and the generated summary from both Model A and Model B.

Here is documentation about ROUGE metrics:
https://github.com/huggingface/evaluate/blob/main/metrics/rouge/README.md

Comparison discussion

Write one to two paragraphs comparing Model A and Model B. Address at least three of the following:

- ROUGE scores — which model performs better quantitatively?
- Convergence speed — which model reached a good validation loss faster?
- Output quality — looking at your examples, which model produces more fluent or accurate summaries?
- Training stability — did either model show loss spikes or instability?
- Your interpretation — did the SSL pre-training in Stage 2 help, hurt, or make no difference? Why?

4.5 Grading Rubric — Track 2

Component	What we look for	Marks	Your score
Stage 1 — Setup	Dataset loaded and described; pretrained checkpoint loaded; parameter count reported	1	
Stage 2 — SSL pre-training	Span corruption implemented; SSL loss curve shows decrease over steps; checkpoint saved	1	
Stage 3 — Fine-tuning (A & B)	Both Model A and Model B fine-tuned; loss curves shown; evaluation metric computed for both	2	
Comparison & analysis	A vs B compared on at least three dimensions; examples shown; interpretation of SSL effect given	1	

Partial credit is available at every level. You will not be penalised for low BLEU or ROUGE scores — what matters is that your pipeline runs correctly and your discussion honestly interprets the results.

5. Shared Requirements (Both Tracks)

5.1 What to Submit

Submit the following files directly to the course portal — do not zip them:

- Your PDF report (filename: StudentID_A3_report.pdf). All figures, plots, tables, and screenshots must be embedded inside the PDF. Do not link to external images.
- Your code files only — .ipynb or .py files. One file per part is fine.
- A plain text README.txt explaining how to run your code and which packages are needed.

Do NOT submit: zip archives, virtual environment folders (.venv, env, __pycache__), or dataset files
Do NOT submit: output folders, model checkpoints, or any file over 20 MB
All plots, grids, loss curves, tables, and example outputs must appear inside the PDF report
Figures embedded in notebooks are acceptable only if the notebook is fully executed (all outputs visible)

5.2 Report Structure

Your report does not need to follow a fixed template, but it should cover:

1. Which track you chose and why (one sentence is fine).
2. Dataset and setup: name, size, resolution or tokenisation, hardware used, approximate training time.
3. Methods: what you implemented, any design choices you made.
4. Results: all required figures, tables, and metrics.
5. Discussion: honest reflection on what worked and what did not.
6. References: cite any code, datasets, or papers you used.

5.3 Compute and Libraries

Any compute platform is acceptable: your own machine, Google Colab, Kaggle, or university resources. Report which GPU (or CPU) you used and the approximate total training time.

Permitted libraries: PyTorch, Hugging Face Transformers and Datasets, torchvision, torchmetrics, tokenizers, and any other standard Python library. If you use a third-party training script or model checkpoint, say so clearly and describe what you changed.